



WinItalPascal

Libreria di utilità per applicazioni VB.NET WinForms

NUGET	V2.0.6	DOWNLOADS	1.1K	LICENSE	MIT	GITHUB PAGES	ONLINE
-------	--------	-----------	------	---------	-----	--------------	--------

Analisi del Codice **GridFilter**

1. Scopo Generale

Il modulo **GridFilter** fornisce due metodi statici (**Shared**) che permettono di filtrare e di ripristinare il filtro di una **DataGridView** in un'applicazione Windows Forms.

Il filtro è applicato in modo dinamico al *data source* della griglia, indipendentemente dal tipo di oggetto di origine (BindingSource, DataView o DataTable).

2. Componenti Chiave

Metodo	Descrizione	Parametri	Operazioni principali
FiltraDgv	Applica un filtro alla griglia.	dgv As DataGridView , filtro As String	1. Determina il tipo di DataSource . 2. Imposta la proprietà di filtro corrispondente.
ResetFiltro	Rimuove qualsiasi filtro impostato.	dgv As DataGridView	1. Determina il tipo di DataSource . 2. Resetta la proprietà di filtro a stringa vuota.

2.1 Tipi di DataSource gestiti

1. **BindingSource** – gestisce la visualizzazione di una collezione di oggetti.
2. **DataView** – vista filtrata di una **DataTable** .
3. **DataTable** – tabella di dati in memoria; il filtro viene applicato tramite **DefaultView.RowFilter** .

3. Flusso di Esecuzione

3.1 Metodo `FiltraDgv`

1. Tentativo di cast a `BindingSource`

```
1 Dim bs = TryCast(dgv.DataSource, BindingSource)
2 If bs IsNot Nothing Then
3     bs.Filter = filtro
4     Exit Sub
5 End If
```

- Se il `DataSource` è un `BindingSource`, la proprietà `Filter` viene impostata con la stringa `filtro` e il metodo termina.

2. Tentativo di cast a `DataView`

```
1 Dim dv = TryCast(dgv.DataSource, DataView)
2 If dv IsNot Nothing Then
3     dv.RowFilter = filtro
4     Exit Sub
5 End If
```

- Se il `DataSource` è un `DataView`, la proprietà `RowFilter` viene impostata.

3. Tentativo di cast a `DataTable`

```
1 Dim dt = TryCast(dgv.DataSource, DataTable)
2 If dt IsNot Nothing Then
3     dt.DefaultView.RowFilter = filtro
4     Exit Sub
5 End If
```

- Se il `DataSource` è una `DataTable`, il filtro viene applicato alla sua `DefaultView`.

4. Fine – Se nessun cast ha avuto successo, il metodo termina senza modificare nulla.

3.2 Metodo `ResetFiltro`

Il flusso è identico a `FiltraDgv`, ma la proprietà di filtro viene impostata a una stringa vuota (`""`) per rimuovere qualsiasi filtro precedente.

4. Possibili Miglioramenti e Considerazioni

Problema / Miglioramento	Descrizione
Gestione di altri tipi di <code>DataSource</code>	Se la griglia utilizza un <code>IEnumerable</code> o un altro tipo di oggetto, il filtro non verrà applicato. Si potrebbe aggiungere un caso di fallback o lanciare un'eccezione informativa.

Problema / Miglioramento	Descrizione
Validazione del filtro	La stringa <code>filtro</code> non viene validata; errori di sintassi (es. colonne inesistenti) genererebbero eccezioni a runtime. Un controllo preliminare o un blocco <code>Try/Catch</code> potrebbe migliorare la robustezza.
Uso di <code>String.Empty</code>	Invece di <code>""</code> , è più leggibile usare <code>String.Empty</code> .
Commenti e documentazione XML	Aggiungere commenti XML ai metodi renderebbe il codice più auto-documentato, utile per IDE e generazione di documentazione.
Separazione della logica di filtro	Se la logica di filtro dovesse diventare più complessa (es. combinazioni di condizioni), si potrebbe estrarre in una classe di helper dedicata.

5. Conclusione

Il codice `GridFilter` è una soluzione semplice e riutilizzabile per filtrare e ripristinare i dati visualizzati in una `DataGridView`.

Gestisce i tre casi più comuni di origine dati e offre un'interfaccia uniforme, consentendo agli sviluppatori di applicare filtri dinamicamente senza preoccuparsi del tipo sottostante di `DataSource`.

Con alcune piccole ottimizzazioni (gestione di errori, supporto per altri tipi di origine dati), il modulo può diventare ancora più robusto e versatile.

Esempio utilizzo

Scopo generale del codice

Il frammento è parte di un form Windows Forms scritto in **VB.NET**.

Si occupa di gestire due eventi dell'interfaccia utente:

1. **Cerca** – evidenzia in una griglia (`FatturaDataGrid`) le celle che contengono il testo digitato dall'utente.
2. **Filtra** – filtra dinamicamente la stessa griglia in base al contenuto di un textbox di ricerca globale (`TxtTutti`).

Componenti principali

Elemento	Descrizione
<code>RjBtnCerca</code>	Bottone che avvia la ricerca.
<code>TxtEvidenzia</code>	TextBox dove l'utente inserisce il termine da evidenziare.

Elemento	Descrizione
<code>TxtTutti</code>	TextBox per filtrare l'intera griglia in tempo reale.
<code>FatturaDataGridView</code>	DataGridView che mostra i dati delle fatture.
<code>dtOriginal</code>	DataTable originale (non filtrata) usata come base per il filtro.
<code>GridUtility</code>	Classe di utilità (non mostrata) che contiene i metodi <code>EvidenziaTesto</code> e <code>FiltraTutti</code> .

Funzioni chiave

- `GridUtility.EvidenziaTesto(grid, filtro)`
Evidenzia tutte le celle del `grid` che contengono la stringa `filtro`. Presumibilmente cambia il colore di sfondo o il font delle celle corrispondenti.
- `GridUtility.FiltraTutti(grid, dataTable, filtro)`
Applica un filtro alla `dataTable` e aggiorna il `grid` per mostrare solo le righe che soddisfano il criterio.
Il filtro è probabilmente una ricerca "contiene" su tutti i campi.

Flusso di esecuzione

1. Evento `RjBtnCerca_Click`

1. Lettura del testo

```
1 Dim filtro As String = TxtEvidenzia.Text.Trim.ToUpper()
```

Si ottiene il contenuto del textbox, si rimuovono spazi iniziali/finali e si converte in maiuscolo per una ricerca case-insensitive.

2. Validazione

Se `filtro` è vuoto, viene mostrato un messaggio di avviso e la sub termina con `Return`.

3. Evidenziazione

Se il filtro è valido, si chiama

```
1 GridUtility.EvidenziaTesto(FatturaDataGridView, filtro)
```

che evidenzia le celle corrispondenti.

2. Evento `TxtTutti_TextChanged`

Ogni volta che l'utente modifica il testo in `TxtTutti`:

```
1 GridUtility.FiltraTutti(FatturaDataGridView, dtOriginal, TxtTutti.Text)
```

Il metodo filtra la griglia in base al nuovo valore, aggiornando la visualizzazione in tempo reale.

Possibili miglioramenti / errori

- **Gestione di eccezioni:** se `GridUtility` lancia eccezioni (es. indice fuori range), sarebbe utile avvolgere le chiamate in `Try...Catch`.
- **Ottimizzazione della ricerca:** convertire `filtro` in maiuscolo solo se il metodo di ricerca è case-insensitive; altrimenti si rischia di perdere la distinzione tra maiuscole/minuscole.
- **Separazione delle preoccupazioni:** la logica di evidenziazione e filtraggio potrebbe essere spostata in un layer di business per rendere il form più “pulito”.
- **Uso di `String.IsNullOrEmpty`:** invece di confrontare con `""`, `String.IsNullOrEmpty(filtro)` è più robusto.

Conclusione

Il codice fornisce una semplice interfaccia di ricerca e filtraggio per una griglia di fatture, sfruttando metodi di utilità per separare la logica di visualizzazione dalla logica di business. È chiaro, leggibile e facilmente estendibile.